PROJECT DOCUMENTATION

Cloud Security Checker

Automated Cloud Security Assessment and Remediation Platform

Project Detail	Description
Project Type	Cloud Computing, Cloud Security & DevOps
Cloud Platform	Amazon Web Services (AWS)
AWS Region	Asia Pacific (Mumbai) — ap-south-1
Operating System	Red Hat Enterprise Linux (RHEL) 10.2
Project Status	Working prototype; validation and improvements pending

Prepared as a technical project reference for training, portfolio development, and interview discussion.

1. Project Overview

Cloud Security Checker is a Python-based platform designed to assess selected security configurations on Amazon EC2 instances and automate specific remediation tasks. A Flask dashboard displays instances and scan results. Python and Boto3 retrieve AWS instance and Security Group information, while Ansible manages selected operating-system security settings over SSH.

The current implementation includes five checks: firewall status, SELinux enforcement, SSH root login configuration, public SSH exposure through Security Groups, and unapproved listening TCP ports.

2. Project Objectives

- Assess selected EC2 instances against predefined security checks.
- Identify selected operating-system and network-access configuration issues.
- Display findings through a web-based dashboard.
- Automate supported remediation tasks using Ansible.
- Remove public SSH ingress rules using AWS Boto3.
- Run a follow-up scan to inspect the resulting security state.
- Practise cloud security, Linux administration, Python automation, AWS IAM, SSH, and Ansible integration.

3. Technologies Used

Technology	Purpose
Amazon EC2	Hosts the dashboard server and target instances.
AWS IAM	Provides the dashboard instance with scoped AWS API permissions.
AWS Security Groups	Controls network access to EC2 instances.
Python	Implements security scanning and remediation logic.
Flask	Serves the web dashboard and scan-result pages.
Boto3	Communicates with AWS EC2 APIs.
Ansible	Automates selected Linux security configurations.
SSH	Connects the dashboard server to target instances.
RHEL	Operating system used by the project servers.
firewalld	Manages the host firewall.
SELinux	Provides mandatory access control.
HTML	Renders the dashboard and scan results.

4. System Architecture

Main Server: Runs Flask, the Python scanner, Boto3 integration, and Ansible commands.

Target servers: Three RHEL EC2 instances reached over private-network SSH.

Component	Recorded configuration
Main server	EC2 instance i-0d70eaee0cebd34d2; private IP 172.31.5.42
Target Server 01	Private IP 172.31.4.205
Target Server 02	Private IP 172.31.7.91
Target Server 03	Private IP 172.31.6.161
AWS Region	ap-south-1

These are the recorded addresses at the time of configuration. Private IP addresses may change depending on the EC2 network configuration.

Workflow: User → Flask Dashboard → Python Scanner → AWS EC2 APIs / SSH Checks → Security Results → User-Initiated Auto Fix → Ansible and Boto3 Remediation → Verification Scan → Updated Results.

The main server uses its attached IAM role for AWS API calls. The dashboard reaches target operating systems using SSH and the configured private key.

5. Security Checks and Remediation

5.1 Firewall Status

The scanner checks whether firewalld is installed and active. PASS means the service is active. FAIL means it is missing, inactive, or could not be verified. The Ansible playbook installs firewalld when necessary, enables and starts it, allows the SSH service, and reloads the configuration.

5.2 SELinux Status

The scanner runs getenforce. PASS means SELinux is Enforcing. Permissive, Disabled, or an unverifiable result is reported as FAIL. The SELinux playbook configures the targeted policy in enforcing mode; disabling SELinux is not part of the intended remediation.

5.3 SSH Root Login

The scanner checks the effective SSH daemon configuration using:

```
sudo sshd -T | grep '^permitrootlogin'
```

PASS means the effective setting is permitrootlogin no. The Ansible playbook updates /etc/ssh/sshd_config, validates the configuration using sshd -t, and reloads sshd. Included configuration files can also affect the effective setting, so post-remediation verification is important.

5.4 AWS Security Group — Public SSH Exposure

Boto3 checks the Security Groups attached to the selected instance. TCP port 22 ingress from 0.0.0.0/0 or ::/0 is flagged as public SSH exposure. The remediation calls RevokeSecurityGroupIngress to remove matching public SSH rules. It does not create replacement rules; administrators must preserve an approved administrative access path.

5.5 Unapproved Listening Ports

The scanner runs ss -tlnH and compares listening TCP ports with the configured approved list: 22 (SSH), 80 (HTTP), and 443 (HTTPS). Other detected listening ports are reported as unapproved by the current policy.

Known limitation: The scanner checks listening sockets, while the current Ansible remediation removes unapproved explicitly allowed TCP ports from firewalld. It does not stop every process listening on an unapproved port. The check and remediation must be aligned and tested before claiming complete port remediation.

6. Application Workflow

- The user opens the Flask dashboard.
- Boto3 retrieves EC2 instance information.
- The dashboard displays available instances and their states.
- The user selects an instance and starts a scan.
- The scanner runs the five security checks.
- The dashboard displays PASS/FAIL results and a score.
- The user selects Auto Fix All to initiate remediation.
- Ansible runs the operating-system remediation playbooks against the selected target.

- Boto3 attempts to remove public SSH ingress rules from the selected instance's attached Security Groups.
- The scanner runs again and displays the resulting status.

> A successful command or API call alone does not prove that a security issue was corrected. The follow-up scan and direct verification are required.

7. Project Directory Structure

Project directory: ~/cloud-security-checker

```
cloud-security-checker/
├── app.py
├── scanner.py
├── sg_scanner.py
├── port_scanner.py
├── remediation.py
├── sg_remediation.py
├── inventory
├── firewall_fix.yml
├── selinux_fix.yml
├── ssh_root_fix.yml
├── ports_fix.yml
├── templates/
│   ├── index.html
│   └── results.html
└── venv/
```

File	Responsibility
app.py	Flask routes, EC2 discovery, dashboard, scan and remediation workflow.
scanner.py	Runs the five security checks and returns results.
sg_scanner.py	Security Group scanning module; integration should be reviewed.
port_scanner.py	Port-scanning module; integration should be reviewed.
remediation.py	Runs Ansible playbooks and calls Security Group remediation.
sg_remediation.py	Removes public SSH ingress rules using Boto3.
inventory	Lists target servers for Ansible.
firewall_fix.yml	Installs/enables firewalld and allows SSH.
selinux_fix.yml	Configures SELinux enforcing mode.
ssh_root_fix.yml	Disables SSH root login and validates the configuration.
ports_fix.yml	Removes unapproved explicitly allowed TCP ports from firewalld.
templates/index.html	Displays EC2 instances and scan links.
templates/results.html	Displays scan results, score, and remediation status.

8. Ansible Inventory and Commands

The inventory stores target private IP addresses and SSH connection settings. The SSH private key must remain outside source control.

```
[targets]
target-server-01 ansible_host=172.31.4.205 ansible_user=ec2-user ansible_ssh_private_key_file=/home/ec2-user/.ssh/key-1.pem
target-server-02 ansible_host=172.31.7.91 ansible_user=ec2-user ansible_ssh_private_key_file=/home/ec2-user/.ssh/key-1.pem
target-server-03 ansible_host=172.31.6.161 ansible_user=ec2-user ansible_ssh_private_key_file=/home/ec2-user/.ssh/key-1.pem
```

Test Ansible connectivity

```
cd ~/cloud-security-checker
ansible -i inventory targets -m ping
```

Run a playbook on one target

```
ansible-playbook -i inventory firewall_fix.yml --limit target-server-01
```

Replace target-server-01 with the appropriate inventory hostname when testing another target.

9. AWS Permissions and Access

The main server uses the attached IAM role CloudSecurityCheckerRole. The recorded permissions are AmazonEC2ReadOnlyAccess and the inline policy CloudSecurityCheckerSGRemediation, which allows ec2:RevokeSecurityGroupIngress. The inline remediation permission does not grant permission to add ingress rules.

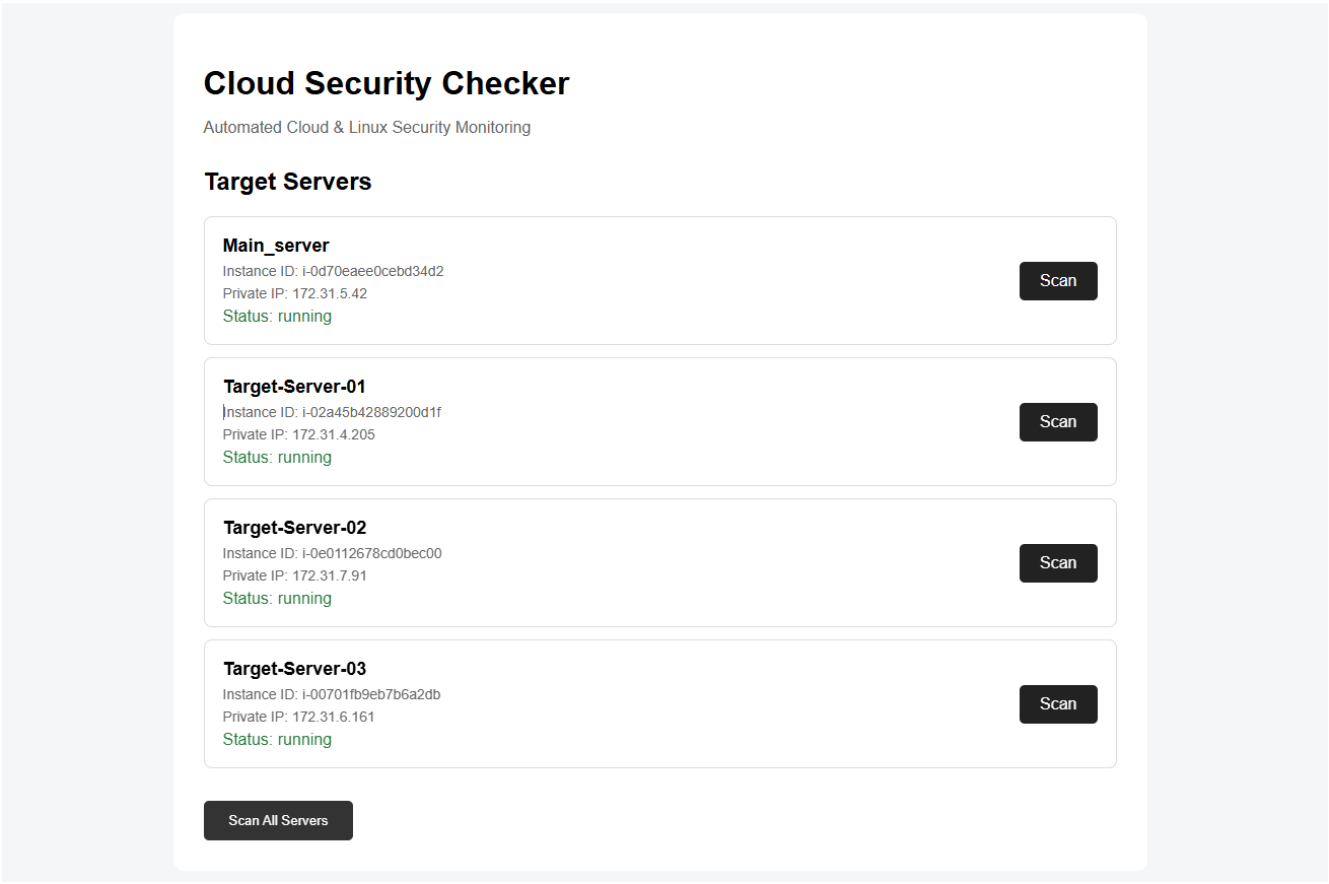
SSH access uses the ec2-user account and the configured private key. Keep private keys and credentials out of the repository. Restrict dashboard access to trusted administrators. Before any public exposure, add authentication, authorization, secure request handling, and suitable network controls.

10. Implementation and Testing Status

- Flask dashboard starts and discovers EC2 instances through Boto3.
- Three target servers are listed in the Ansible inventory.
- SSH connectivity to the target servers has been tested.
- Ansible ping has returned successful results for all three targets.
- All five security checks are integrated into the scan workflow.
- Remediation modules are connected to the dashboard.
- The dashboard has reported successful remediation module results for all three targets.
- The current workflow performs a scan after remediation.

> Validation status: A controlled failure-and-recovery test has not yet been documented. The current results establish prototype functionality, not production readiness.

Dashboard


Cloud Security Checker
Automated Cloud & Linux Security Monitoring
Target Servers
Main_server
Instance ID: i-0d70eaee0cebd34d2
Private IP: 172.31.5.42
Status: running
Scan
Target-Server-01
Instance ID: i-02a45b42889200d1f
Private IP: 172.31.4.205
Status: running
Scan
Target-Server-02
Instance ID: i-0e0112678cd0bec00
Private IP: 172.31.7.91
Status: running
Scan
Target-Server-03
Instance ID: i-00701fb9eb7b6a2db
Private IP: 172.31.6.161
Status: running
Scan
Scan All Servers

Before fix

Security Scan Results

Instance ID: i-00701fb9eb7b6a2db

Security Score: 4 / 5

Firewall
FAIL
firewalld is not installed

SELinux
PASS
SELinux is enforcing

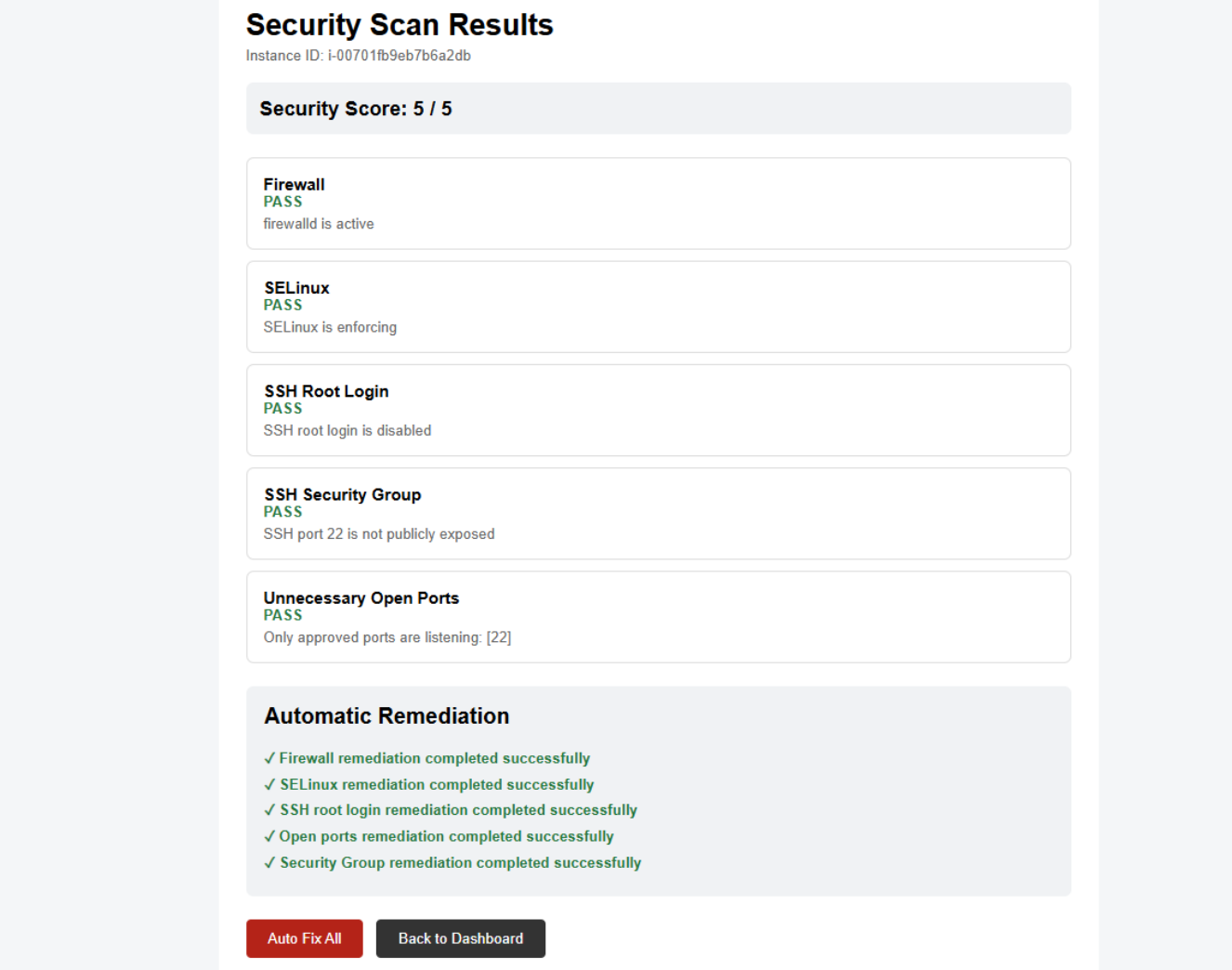
SSH Root Login
PASS
SSH root login is disabled

SSH Security Group
PASS
SSH port 22 is not publicly exposed

Unnecessary Open Ports
PASS
Only approved ports are listening: [22]

Auto Fix All | Back to Dashboard

After fix



11. Conclusion

Cloud Security Checker is a working prototype that combines AWS EC2 discovery, Python-based security checks, a Flask dashboard, Ansible-based host remediation, and Boto3-based Security Group remediation. It demonstrates practical integration of cloud infrastructure, Linux security, and automation. Controlled failure testing, improved port remediation, dashboard hardening, and portfolio documentation remain necessary before presenting it as a fully validated security tool.

End of document